University of Dhaka
Department of Computer Science and Engineering

Software Design Document

Academic Operations Suite

Curriculum • Class Routine • Exam Routine
Invigilation Assignment • Thesis Management

CSE 4113 — Internet Programming Lab

Submitted By:

Team: Newbie

Marufur Rahman Mithu (19)
Tonmoy Das (22)
Manobendra Biswas (35)
Faisal Ahmed (54)

Course Instructors:

Prof. Dr. Md. Mamun Or Rashid
Professor, Dept. of CSE, University of Dhaka

Md. Fahim Arefin
Lecturer, Dept. of CSE, University of Dhaka

Mr. Md. Ahasanul Alam
Lecturer, Dept. of CSE, University of Dhaka

April 9, 2026

Contents

1 Team and Project Information **2**

2 System Overview **2**

3 Architecture Design **3**

3.1 High-Level Architecture 3

3.2 Component Breakdown 4

3.3 Key Design Decisions 5

4 Data Design **6**

5 Core Flow Design **8**

6 Security and Access Design **11**

7 Implementation Plan **12**

8 Testing Plan (Design Level) **13**

Appendix: API Endpoint Summary **15**

1 Team and Project Information

Field	Value
Team	Newbie
Project Name	Academic Operations Suite
Repository URL	https://github.com/manob-endra/AcademicOperationsSuit
Commit / Tag	`v0.2-sdd`
Date	April 9, 2026
Related SRS Version	0.2 (April 8, 2026)

2 System Overview

Problem Summary

The Department of Computer Science and Engineering at the University of Dhaka currently manages all academic scheduling — class routines, exam timetables, invigilation assignments, thesis supervision, and curriculum transitions — through manual, spreadsheet-driven processes. These processes are time-consuming, highly error-prone, and produce no single authoritative source of truth. Scheduling conflicts (double-booked rooms, teacher overlaps, simultaneous exams for the same student section) go undetected until they cause real disruptions. Invigilation duties are distributed unevenly, and students who switch curriculum versions cannot easily identify which courses they still need to complete.

Target Users

User Group	Description	Primary Needs
Super Admin	Chairman or designated officer	Full system control, role management
Admin / Officer	Staff managing scheduling ops	Create/publish routines, manage exams and invigilation
Teacher / Faculty	Instructors assigned to courses	View schedules, declare availability, manage thesis
Student	Enrolled students	View class/exam routines, missing course finder

Project Goals

- Automate class and exam routine generation with real-time conflict detection.

- Provide multi-version curriculum management with side-by-side comparison.
- Fairly distribute invigilation duties while respecting teacher availability constraints.
- Publish schedules to read-only student and faculty portals.
- Identify missing courses for students switching curriculum versions.
- Manage thesis supervision assignment and defense scheduling.
- Export all schedules and rosters to PDF and Excel formats.

Out of Scope

- Student grading, transcript management, or GPA calculation.
- Learning management features (assignments, lecture notes, video content).
- Native mobile applications (iOS / Android) — web-responsive design only.
- Third-party university ERP integration (other than optional billing API).
- Multi-department or multi-university deployment.
- Dark mode (deferred to a future version).

3 Architecture Design

3.1 High-Level Architecture

The system follows a **Single-Page Application (SPA) + RESTful API** architecture deployed entirely on free-tier cloud infrastructure. The three primary tiers are:

1. **Frontend (React.js v18+):** A client-side SPA built with React and styled using Tailwind CSS. All user interactions happen through this layer. The frontend communicates exclusively with the backend via REST API calls over HTTPS.
2. **Backend (Node.js + Supabase):** A RESTful API server (mandated by the course to use Supabase as the backend-as-a-service platform) that exposes all business logic endpoints. It handles authentication (Google OAuth 2.0 + JWT), conflict-detection algorithms, invigilation assignment logic, and data persistence.
3. **Database (PostgreSQL via Supabase):** All persistent data is stored in a PostgreSQL database managed by Supabase. Supabase also provides Row-Level Security (RLS) policies as an additional authorization layer.

Data Flow Summary: The browser sends HTTPS requests to the API server. The API authenticates the request via the JWT present in an `httpOnly` cookie, applies RBAC checks, executes business logic (including conflict detection), and queries / mutates the PostgreSQL database. Responses are returned as JSON to the React frontend, which re-renders accordingly. Export operations generate PDF/Excel files on the server and stream them to the browser.

External Integrations:

- **Google OAuth 2.0** — for authentication; domain filtering restricts access to `@cs.du.ac.bd` and `@cse.du.ac.bd` accounts only.
- **Team Formula1 Billing API** (optional, Could Have priority) — integrated via an adapter pattern so the core system functions without it.
- **Vercel** (frontend hosting) and **Render / Railway** (backend hosting) — free-tier cloud deployment.

3.2 Component Breakdown

Component	Responsibility	Inputs	Outputs	Notes
Auth Module	Google OAuth flow, JWT issuance, session management, RBAC enforcement	Google token, domain, user credentials	JWT cookie, user role, session token	httpOnly cookie; 24h expiry + refresh
Curriculum Manager	CRUD for curriculum versions, versioning on edit, publish/archive lifecycle, comparison view	Curriculum data (version, courses), admin actions	Curriculum records, diff output	Auto-creates minor version on edit of published curriculum
Class Routine Engine	Routine creation, teacher/room/sectior conflict detection, drag-and-drop editing, publish	Slots (day, time, teacher, room, course), routine metadata	Draft/published routine, conflict report	Conflict check must run within 1 s of slot assignment
Exam Routine Engine	Exam schedule creation, multi-sitting support, conflict detection, publish	Course list, dates, rooms, capacity	Exam schedule, conflict report	Blocks save on conflict; back-to-back shows warning
Invigilation Engine	Pool definition, workload-balanced auto-assignment, constraint enforcement, roster export	Exam routine, teacher pool, unavailability constraints	Invigilation roster, duty counts, unresolved-slot report	Duty counts within 1 of mean; red banner for unresolved slots
Thesis Manager	Thesis registration, supervisor assignment, defense scheduling, conflict detection	Student ID, title, supervisor, defense date	Thesis record, conflict warnings	Should Have priority

Continued on next page...

Component	Responsibility	Inputs	Outputs	Notes
Missing Course Finder	Equivalency mapping management, missing course calculation, credit summary, PDF export	Student profile, completed courses, target curriculum	Missing course list, remaining credits, PDF report	Should Have priority
Published Portals	Read-only student and faculty schedule views, PDF download	Published routine/exam data	Rendered timetable (grid + list), PDF	Semester and section filters; printable PDF
API Layer	RESTful endpoints, request validation, rate limiting, Swagger docs	HTTP requests with JWT	JSON responses, HTTP status codes	Documented at `/api/docs`; 100 req/IP/min rate limit
Export Service	Generate PDF and Excel files for routines, exam schedules, invigilation rosters	Published data entities	`.pdf` / `.xlsx` downloads	Streams files directly to client

3.3 Key Design Decisions

Decision	Options Considered	Chosen Option	Reason
Authentication strategy	Password-based login, Google OAuth 2.0, university SSO	Google OAuth 2.0 with domain filtering	Course mandate; eliminates password storage; enforces DU domain restriction natively
Backend platform	Custom Express.js + PostgreSQL, Firebase, Supabase	Supabase (PostgreSQL + BaaS)	Course mandate; provides built-in auth helpers, RLS, and managed PostgreSQL on free tier
Conflict detection approach	Rule-based (synchronous), graph-based scheduling, constraint solver	Rule-based synchronous detection	Simpler to implement within one semester; deterministic; meets the 5 s performance requirement; can be upgraded later

Decision	Options Considered	Chosen Option	Reason
Frontend framework	Vue.js, Angular, React.js	React.js v18+	Course mandate; large ecosystem; component model suits the timetable grid and drag-and-drop UI
Curriculum versioning strategy	Overwrite in place, full copy on every edit, minor version on edit of published	Minor version (e.g., v2023.1) auto-created on edit of published curriculum	Preserves history while keeping storage efficient; published version remains unmodified
Invigilation assignment algorithm	Manual only, greedy workload-balanced, full ILP solver	Greedy workload-balanced auto-assignment	Achieves the "within 1 of mean" fairness requirement; fast enough for typical pool sizes; ILP unnecessary at this scale
Export format	Client-side generation, server-side generation, external service	Server-side generation (streamed to client)	Consistent formatting across browsers; PDF fidelity; keeps client bundle small
Deployment	Self-hosted VPS, Heroku, Vercel + Render/Railway	Vercel (frontend) + Render/Railway (backend) + Supabase (DB)	Free-tier constraint; proven for React SPAs and Node.js APIs; Supabase manages DB hosting

4 Data Design

Core Entities

- **User** — Authenticated system user with a role and Google identity.
- **Curriculum** — Versioned degree program containing an ordered course list.
- **Course** — Individual course record (code, title, credits, type, semester) embedded within a curriculum.
- **Room** — Physical classroom or lab with capacity and type.
- **ClassRoutine** — Weekly class timetable per academic year, semester, and section.
- **RoutineSlot** — A single time-slot entry within a class routine (day, time, teacher, room, course).
- **ExamRoutine** — Semester exam schedule with room and time assignments.

- **ExamSlot** — Individual exam event (course, date, time, duration, rooms, invigilators).
- **InvigilationDuty** — Assignment record linking a teacher to an exam slot.
- **Thesis** — Student thesis/project record with supervisor and defense scheduling.
- **CurriculumMapping** — Equivalency record linking courses across curriculum versions.

Key Relationships

- **User → Curriculum (1:many):** An Admin creates and edits curricula; `createdBy` references User.
- **Curriculum → Course (1:many, embedded):** Each curriculum embeds its course list for atomic updates; courses are not shared across curricula by reference.
- **ClassRoutine → RoutineSlot (1:many, embedded):** Each slot stores `teacherId` (ref: User) and `roomId` (ref: Room) as foreign keys.
- **ExamRoutine → ExamSlot (1:many):** Each exam slot embeds `rooms[]` and `invigilators[]` as ID references.
- **InvigilationDuty joins User and ExamSlot (many:many):** One duty record per teacher–exam assignment; enables efficient per-teacher and per-exam queries.
- **Thesis → User (multiple roles):** `studentId`, `supervisorId`, `coSupervisorId`, and `committeeMembers[]` are all references to User.
- **CurriculumMapping links Curriculum + Course pairs:** `fromCurriculumId.fromCourseCode` maps to `toCurriculumId.toCourseCode`.

Constraints (Unique, Nullability, Validation)

Field	Type	Required	Validation / Constraint
`User.email`	String	Yes	Must match `/@(cs|cse)duacbd$/`; max 100 chars; unique
`User.role`	Enum	Yes	One of: `student`, `teacher`, `admin`, `superadmin`
`Curriculum.versio`	String	Yes	Format: 4-digit year or year.minor; unique per program
`Course.code`	String	Yes	Format: `CSE XXXX`; unique within curriculum
`Course.credits`	Number	Yes	Positive; typically 3.0 theory, 1.5 lab
`Course.type`	Enum	Yes	One of: `theory`, `lab`, `project`
`Course.semester`	Integer	Yes	Integer 1–8 inclusive
`Room.capacity`	Integer	Yes	Positive integer; maximum 500
`RoutineSlot.day`	Enum	Yes	One of: Monday–Friday

Field	Type	Required	Validation / Constraint
`RoutineSlot.startTime`	String	Yes	HH:MM (24h); within 08:00–18:00; before `endTime`
`ClassRoutine.stat`	Enum	Yes	One of: `draft`, `published`
`ExamSlot.duration`	Integer	Yes	Exam duration in minutes; 30–240
`Thesis.defenseDat`	Date	No	Must be a future date when scheduling; ISO 8601
`ClassRoutine.(year,semester,section)`	Composite	Yes	Unique composite index
`ExamRoutine.(year`	Composite	Yes	Unique composite index

Data Lifecycle Notes

- **Curriculum:** Created as `draft` → promoted to `published` (student-visible) → optionally moved to `archived` (read-only; cannot be used in scheduling). Editing a published curriculum auto-creates a minor version, preserving the original.
- **ClassRoutine / ExamRoutine:** Exist as `draft` until all conflicts are resolved, then promoted to `published` by an Admin. Published routines are immutable except via a new draft cycle.
- **InvigilationDuty:** Created by auto-assignment or manual override; status transitions through `assigned` → `confirmed` → `swapped`. All slots must be assigned before the roster can be published.
- **Thesis:** Created as `registered` → `defended` → `completed`.
- **Backups:** Supabase automated daily backups; point-in-time recovery for the last 7 days.

5 Core Flow Design

Flow 1: Admin Generates and Publishes a Class Routine

Trigger: An Admin navigates to the Class Routines module and clicks *Create New*.

1. Admin selects academic year, semester, and section; the system creates a new ClassRoutine record in `draft` status.
2. Admin adds courses and assigns teachers and rooms to time slots via the timetable grid UI.
3. On each slot assignment, the frontend sends a conflict-check request (`POST /api/v1/routines/clas` to the backend.
4. The backend rule-based engine checks for: (a) room double-booking, (b) teacher double-booking, (c) same-section time overlap, and (d) consecutive-period overload. Conflicts are returned as a structured list.
5. Conflicted slots are highlighted in red in the UI within 1 second; the live Conflict Report panel updates immediately.

6. Admin resolves all conflicts (reassigning teachers, rooms, or time slots, including via drag-and-drop).
7. Admin clicks *Publish*; the backend verifies the conflict count is zero, then changes status to `published`.
8. The published routine becomes visible in the student and faculty read-only portals.

Success Outcome: Class routine status is `published`; conflict report is empty; routine appears in student and faculty portals; Admin sees a success toast notification.

Failure / Edge Cases:

- *Publish attempted with unresolved conflicts:* Publish button is disabled; tooltip reads "Resolve all conflicts before publishing."
- *Teacher has no available slots:* Admin is warned and may choose a different teacher.
- *Conflict cannot be immediately resolved:* Admin saves as `draft` and returns later.
- *Network timeout during conflict check:* UI shows an error banner; slot is not committed.

Flow 2: Admin Runs Workload-Balanced Invigilation Auto-Assignment

Trigger: Admin navigates to the Invigilation Assignment module, selects a finalized exam routine, and clicks *Auto-Assign Invigilators*.

1. The backend retrieves all ExamSlots for the selected exam routine and the defined invigilator pool (with faculty weights for full-time/part-time).
2. For each exam slot (ordered by date/time), the greedy assignment algorithm filters the pool to eligible teachers: those who are (a) not marked unavailable, (b) not teaching at that time (cross-checked against published class routines), and (c) not already assigned to another room in the same exam slot.
3. Among eligible teachers, the algorithm picks the teacher(s) with the lowest current duty count to maintain workload balance (duty counts within 1 of the mean).
4. An InvigilationDuty record is created for each assignment.
5. If no eligible teacher remains for a slot, the slot is flagged as unresolved: a persistent red banner appears ("Warning: N exam slot(s) could not be assigned"); the affected slots are highlighted red in the Unresolved Slots section; the Publish button is disabled.
6. For each unresolved slot, the Admin is offered three inline resolution actions: (a) *Expand Pool* — add a previously excluded faculty member; (b) *Manual Override* — assign any faculty via dropdown, bypassing pool restrictions; (c) *Reschedule* — navigate to the Exam Routine editor to move the slot to a time when eligible invigilators are free.
7. Once all slots are resolved, the Publish button re-enables and the red banner clears.
8. Admin exports the finalized roster as PDF or Excel.

Success Outcome: All exam slots have at least one assigned invigilator; duty distribution is within 1 of the mean across the pool; roster is exportable and publishable.

Failure / Edge Cases:

- *Entire pool unavailable for a slot:* System enters the unresolved-slot flow described in step 5–6 above.
- *Pool is empty:* Admin is warned before running auto-assign; the run is blocked.
- *Admin swaps an assignment manually:* System re-checks constraints and warns if the new assignment creates a conflict.
- *Exam routine modified after assignment:* Existing assignments are invalidated and admin must re-run auto-assign.

Flow 3: Student Uses the Missing Course Finder After Curriculum Shift

Trigger: A student (or Admin on behalf of a student) opens the Missing Course Finder.

1. Student enters their Student ID, selects their original curriculum version (e.g., Curriculum 2016), and inputs or selects their list of completed courses and grades.
2. Student selects the target curriculum version (e.g., Curriculum 2023) and clicks *Calculate*.
3. The backend loads the CurriculumMapping records between the two versions and maps each completed course to its equivalent in the target curriculum (if a mapping exists).
4. Completed courses with no equivalency mapping are flagged as *unmapped*; the Admin is prompted to define the mapping.
5. The system computes: (a) mapped and satisfied requirements, (b) missing courses (requirements in the target curriculum not satisfied by any mapped completion), and (c) total remaining credits.
6. Results are displayed: missing course list, mapped equivalents, and a credit summary.
7. Student can export the analysis as a formatted PDF for official advising use.

Success Outcome: Student sees a complete list of remaining required courses and credits; PDF report is downloadable.

Failure / Edge Cases:

- *No mapping defined:* Unmapped courses are flagged and excluded from the calculation; Admin is alerted.
- *Partial equivalency (e.g., a 3-credit course maps to two 1.5-credit courses):* Handled as an open question; current design flags these for manual review.
- *Invalid Student ID:* Form validation rejects the input before submission.

6 Security and Access Design

Authentication Approach

All authentication is handled exclusively via **Google OAuth 2.0**. No passwords are ever stored in any form. The OAuth flow is initiated at `GET /api/v1/auth/google`; the callback at `GET /api/v1/auth/callback` validates the Google ID token, verifies the email domain (only `@cs.du.ac.bd` and `@cse.du.ac.bd` are permitted), and issues a signed JWT stored in an `httpOnly` cookie. The JWT expires after 24 hours; a refresh token extends the session. Logout at `POST /api/v1/auth/logout` invalidates the JWT.

Authorization Model (Roles / Permissions)

Role	Assigned To	Permissions
Super Admin	Department chairman / designated officer	Full CRUD on all modules, user role management, system configuration
Admin / Officer	Academic scheduling staff	Create/edit/publish routines, assign invigilation, manage thesis
Teacher / Faculty	`@cse.du.ac.bd` accounts (auto-mapped)	View own schedule, declare availability, update thesis status
Student	`@cs.du.ac.bd` accounts (auto-mapped)	Read-only access to published portals, missing course finder

All API endpoints and UI routes enforce Role-Based Access Control (RBAC). Unauthorized access returns HTTP 403; the UI hides menu items and routes not accessible to the current role. Supabase Row-Level Security (RLS) policies provide a second enforcement layer at the database level.

Sensitive Data Handling

- All data in transit is encrypted using HTTPS (TLS 1.2 or higher); no HTTP endpoints are exposed. Target: SSL Labs grade A or A+.
- JWTs are stored in `httpOnly` cookies (not `localStorage`) to mitigate XSS token theft.
- All user inputs are validated on both the frontend and the backend; inputs are sanitized against XSS and SQL injection. OWASP ZAP scans will be run to verify no vulnerabilities.
- API endpoints enforce rate limiting of 100 requests per IP per minute; requests beyond the limit return HTTP 429.

- No student grades, financial data, or government identifiers are stored; the system handles only scheduling and curriculum data.

Audit / Logging Plan

- All authentication events (login, logout, domain rejection, role elevation) are logged with timestamp, user email, and IP address.
- All publish and archive actions on routines, curricula, and rosters are logged with the acting user's ID and timestamp.
- Failed authorization attempts (HTTP 403 responses) are logged for anomaly detection.
- Logs are retained in the Supabase backend for a minimum of one semester; export to external logging service is a future enhancement.

7 Implementation Plan

Sprint 1 Completed Features

- Google OAuth 2.0 login with domain filtering (`@cs.du.ac.bd` / `@cse.du.ac.bd`).
- Domain-based automatic role mapping (Student / Teacher on first login).
- JWT session management with `httpOnly` cookies, 24-hour expiry, and refresh token support.
- Super Admin user role management panel.
- RBAC enforcement on all API endpoints and UI routes.
- Curriculum CRUD (create, edit, version, publish, archive).
- Basic class routine creation (draft state, room and teacher slot assignment).
- Initial real-time conflict detection for room and teacher double-booking.

Pending Features for Sprint 2

- Complete conflict detection (section overlap, consecutive-overload, conflict report view).
- Drag-and-drop slot reassignment with live conflict re-check.
- Class routine publish flow and student/faculty portal.
- Exam routine generation with multi-sitting support and conflict detection.
- Invigilation auto-assignment with workload balancing, unresolved-slot warning UI, and all three resolution paths.
- Invigilation roster export (PDF and Excel).
- Curriculum comparison view (side-by-side diff).

- Missing Course Finder (equivalency mapping + calculation).
- Thesis supervision and defense scheduling module.
- PDF and Excel export for class routines, exam routines, and invigilation rosters.
- Swagger/OpenAPI 3.0 documentation at `/api/docs`.

Technical Risks

Risk	Prob.	Impact	Notes
Scope creep with 8+ modules	High	High	Apply MoSCoW strictly; weekly scope review
Supabase free-tier connection/storage limits	Med	Med	Monitor metrics; optimize indexes; upgrade tier if needed
Conflict detection complexity causing performance delays	Med	Med	Start with simple rule-based; optimize incrementally; 5 s timeout fallback
Google OAuth domain misconfiguration	Med	High	Thorough domain filter testing; manual admin override for edge cases
Team member unavailability	Med	Med	Modular task ownership; cross-training on critical modules
Data loss during development	Low	High	Git version control from day one; Supabase automated backups

Mitigation Plan

- MoSCoW prioritization is enforced: all Must Have features take precedence; Should Have and Could Have features are deferred if timeline is at risk.
- Weekly team syncs to review progress, redistribute tasks, and surface blockers early.
- The adapter pattern is used for the optional billing API integration so the core system never depends on it.
- End-to-end testing of the OAuth domain filter will be completed in Sprint 2 before any other authentication-dependent feature is finalized.

8 Testing Plan (Design Level)

Unit Testing Targets

- **Conflict detection engine:** Individual rule functions for room double-booking, teacher double-booking, section overlap, and consecutive-overload — tested with a matrix of pass/fail slot combinations.

- **Invigilation assignment algorithm:** Workload-balance logic (duty count within 1 of mean), eligibility filtering (unavailability, teaching overlap, double-booking) — tested with synthetic pool data.
- **Missing Course Finder calculation:** Mapping resolution, missing course identification, and credit summation — tested against a manually verified dataset.
- **JWT utilities:** Token issuance, expiry, refresh, and invalidation logic.
- **Input validation:** Email domain regex, course code format, time range constraints, enum membership.

Integration Testing Targets

- **Auth flow:** Valid domain login succeeds; invalid domain is rejected; role is correctly auto-assigned; JWT cookie is set; logout invalidates token.
- **Class routine conflict detection endpoint** (`POST /api/v1/routines/class/check-conflicts`): Returns correct conflict list for known conflict scenarios.
- **Invigilation auto-assignment endpoint** (`POST /api/v1/invigilation/assign`): Returns correct roster; triggers unresolved-slot response when pool is fully constrained.
- **Publish endpoints:** Blocked when conflicts or unresolved slots exist; succeeds and updates status when all conditions are met.
- **RBAC enforcement:** Student and Teacher roles cannot access Admin-only routes; HTTP 403 returned.
- **Export endpoints:** PDF and Excel downloads produce correctly formatted files for class routines, exam routines, and invigilation rosters.

End-to-End (E2E) Testing Targets

The test cases from the SRS Initial Manual Test Plan (Section 11.2) serve as the E2E test suite baseline. Key E2E scenarios:

- **TC-T01-001/002:** Google login with valid / invalid domain.
- **TC-T03-002/003:** Room and teacher conflict detection during routine building.
- **TC-T04-002:** Exam conflict blocked on save.
- **TC-T05-004/005:** Unresolvable invigilation slot — red banner, Publish disabled, all three resolution paths functional.
- **TC-T07-001:** Missing course calculation accuracy.
- **TC-T08-001/002:** Student portal routine view and PDF download.

Known Hard-to-Test Areas

- **Google OAuth flow:** Requires a live Google account with a DU-domain email; cannot be fully mocked in unit tests. Mitigation: use test accounts provisioned by the department.

- **Drag-and-drop conflict re-check timing:** The requirement that conflict check runs within 1 second of a drag-and-drop move is hard to assert reliably in automated tests. Mitigation: manual timing tests; debounce mechanism in frontend code.
- **Concurrent conflict detection:** Race conditions when two Admins edit the same routine simultaneously are not fully addressed in this version; concurrency is treated as a known risk.
- **PDF visual fidelity:** Automated assertion of PDF layout correctness is difficult. Mitigation: visual regression testing with a reference PDF.
- **Free-tier resource limits:** Load testing at 100 concurrent users may be constrained by Supabase / Render free-tier connection limits; testing will be performed during off-peak hours.

Appendix: API Endpoint Summary

Method	Endpoint	Auth Role	Description
GET	`/api/v1/auth/google`	Public	Initiate Google OAuth flow
GET	`/api/v1/auth/callback`	Public	OAuth callback handler
POST	`/api/v1/auth/logout`	Any	Logout and invalidate JWT
GET	`/api/v1/users`	Admin	List all users with roles
PUT	`/api/v1/users/:id/role`	Super Admin	Update user role
GET	`/api/v1/curricula`	Any	List all curriculum versions
POST	`/api/v1/curricula`	Admin	Create new curriculum version
GET	`/api/v1/curricula/:id`	Any	Get curriculum details with courses
PUT	`/api/v1/curricula/:id`	Admin	Edit curriculum (auto-versions if published)
GET	`/api/v1/routines/class`	Any	List class routines
POST	`/api/v1/routines/class`	Admin	Create class routine draft
POST	`/api/v1/routines/class/check-conflicts`	Admin	Run conflict detection
PUT	`/api/v1/routines/class/:id/`	Admin	Publish class routine
GET	`/api/v1/routines/exam`	Any	List exam routines
POST	`/api/v1/routines/exam`	Admin	Create exam routine
GET	`/api/v1/invigilation`	Admin	List all invigilation duties
POST	`/api/v1/invigilation/assign`	Admin	Auto-assign invigilators with balancing
GET	`/api/v1/thesis`	Admin/Teacher	List thesis records
POST	`/api/v1/thesis`	Admin	Register new thesis
POST	`/api/v1/missing-courses`	Any Auth	Calculate missing courses for a student

Method	Endpoint	Auth Role	Description
GET	`/api/v1/rooms`	Any	List available rooms and labs
POST	`/api/v1/rooms`	Admin	Create room or lab
GET	`/api/v1/export/routine/:id`	Any Auth	Export routine as PDF or Excel